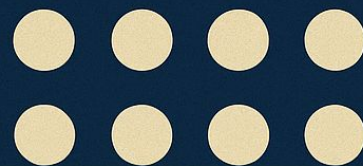
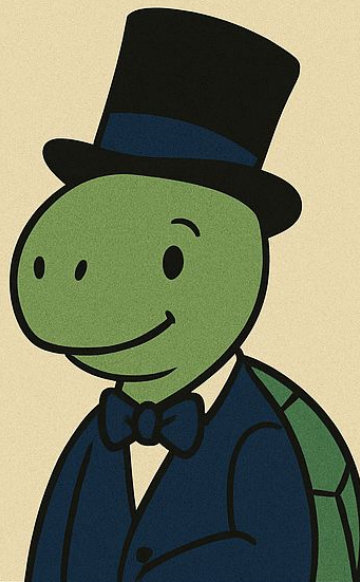


ROS WORKSHOP

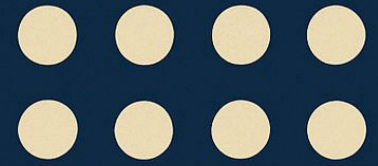


ROS WORKSHOP



16/05/25 - B15 LM4

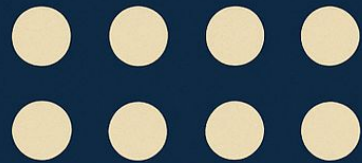
Scheduling



- **6th May** - Introduction to the software ROS and first examples
- **12th May** - Introduction to the GNSS module and how to use GNSS data inside ROS
- **16th May** - Introduction to the navigation problem and examples



Github link

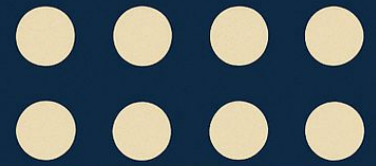


You can download all the example/slides from here:

<https://github.com/LucaAttorrese/ROS-Workshop/>



The Navigation Problem

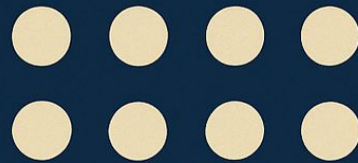


Last time we've seen how **sensors** are used in robotics to solve the **localization problem**.

To have a fully autonomous mobile robot, we need to address also the **navigation one**.



SLAM

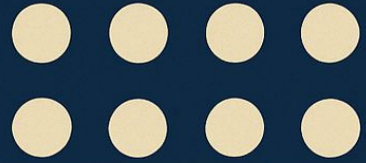


Usually, for the localization we use:

Simultaneous
Localization
And
Mapping



SLAM



Simultaneous: in real time.

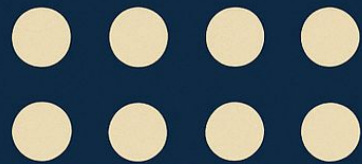
Localization: get robot's position and orientation within its environment.

And: while.

Mapping: autonomously creating a map of its environment.



Navigation problem



Navigation: autonomously navigate to desired locations while avoiding obstacles.

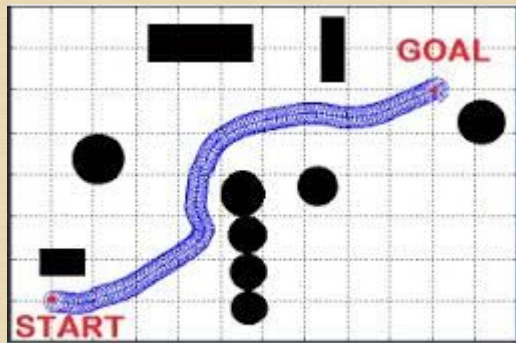
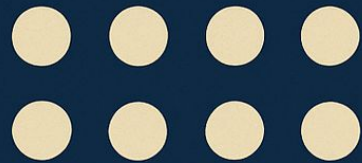


FIG. 4. Mobile Robot Navigation, $k_{rep}=0.01$



Ex. 3.1

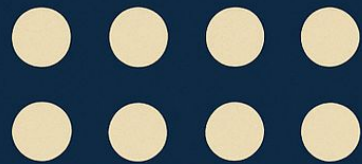


What we'll do today in ROS:

- **SLAM** (create a map while moving around)
- **Navigation** (autonomously move the robot inside the map)



Ex. 3.1 (mapping)

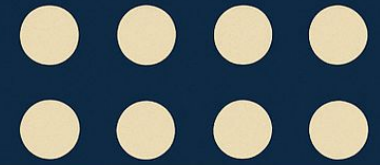


Some new tools we're using today:

- Gazebo
- Nav2 package
- **(Turtlebot)**



Ex. 3.1 (mapping)



TURTLESIM

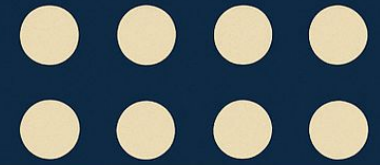


TURTLEBOT

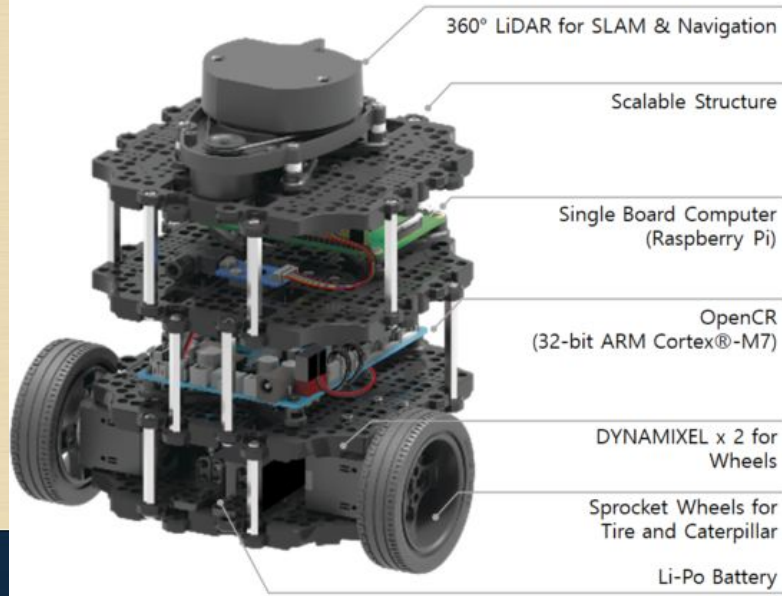
imgflip.com



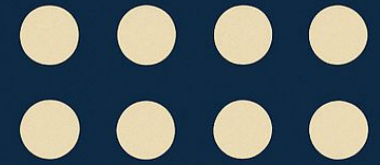
Ex. 3.1 (mapping)



TurtleBot3 Burger



Ex. 3.1: Gazebo



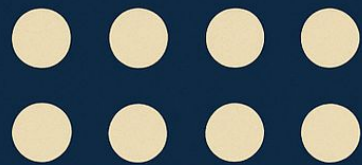
2D/3D robot simulator that can be easily integrated with ROS



GAZEBO



Ex. 3.1: Gazebo (install)



As always:

- Open powershell
- Enter wsl (wsl)
- Source ros
(source /opt/ros/jazzy/setup.bash)



Ex. 3.1: Gazebo (install)

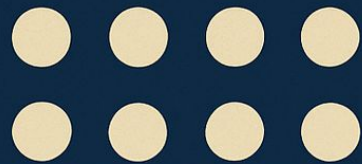
As always:

- Open powershell
- Enter wsl (wsl)
- Source ros

(source /opt/ros/jazzy/setup.bash)



Ex. 3.1: Gazebo (install)



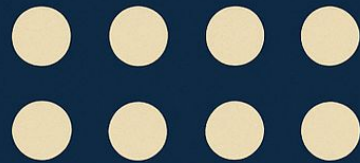
Install Gazebo:

```
sudo apt update
```

```
sudo apt install ros-${ROS_DISTRO}-ros-gz
```



Ex. 3.1: Nav2



ROS Navigation package

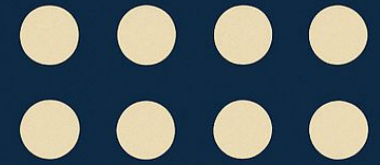
To install:

```
sudo apt install ros-jazzy-navigation2 ros-jazzy-nav2-bringup
```

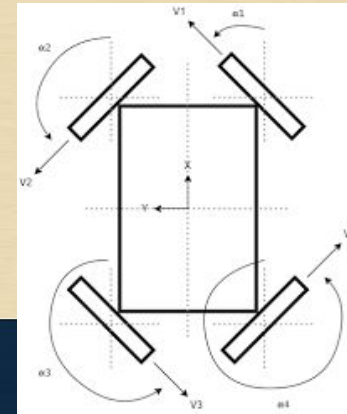
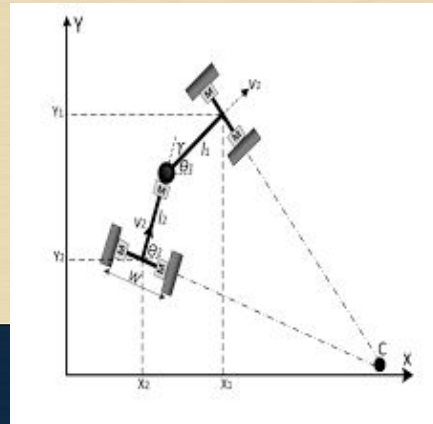
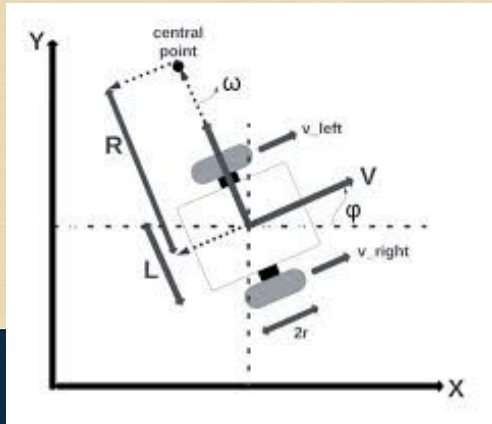
(it will take some minutes)



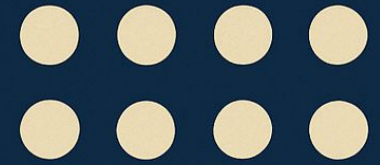
Ex. 3.1: Nav2



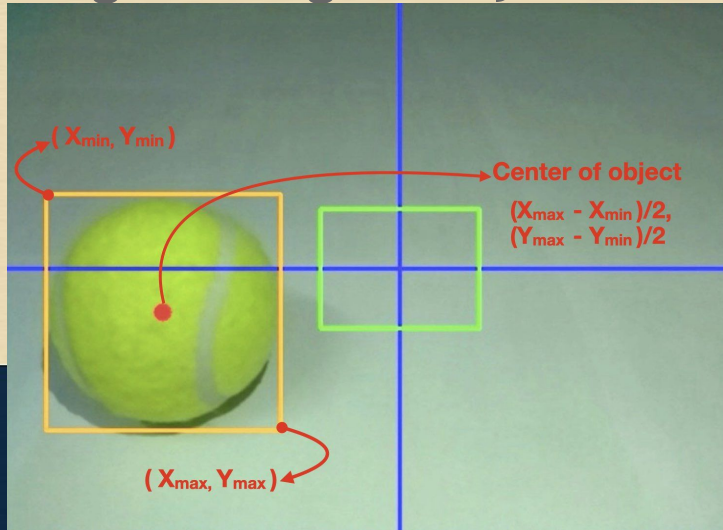
NAV2 allows for mobile robots to **navigate through complex environments** to complete user-defined application tasks with nearly **any class of robot kinematics**.



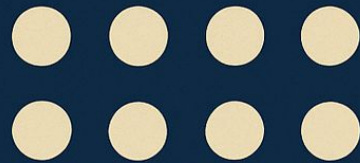
Ex. 3.1: Nav2



Not only can it move from Point A to Point B, but **it** can have intermediary poses, and represent other types of tasks like object following, complete coverage navigation, and more.



Ex. 3.1: Nav2



Nav2 is a **production-grade** and high-quality navigation framework **trusted by 100+ companies worldwide.**

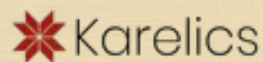
kîwîbot



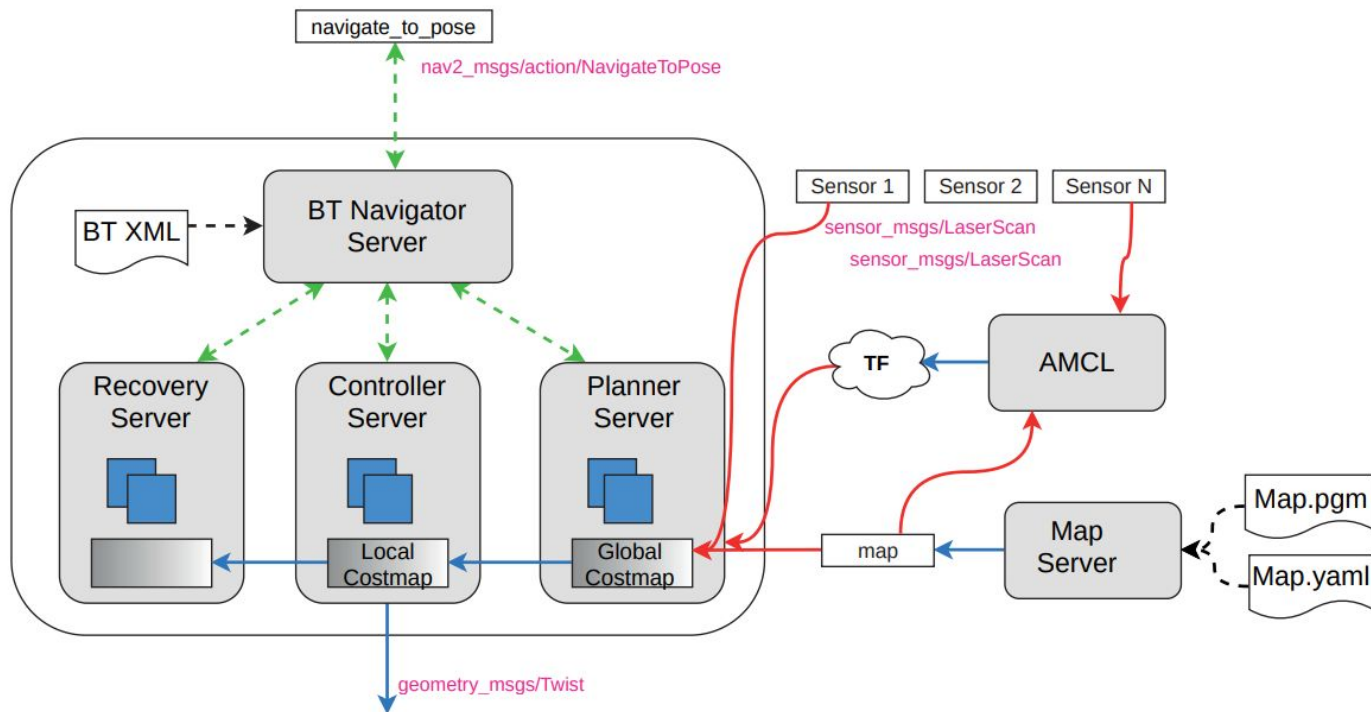
TOYOTA



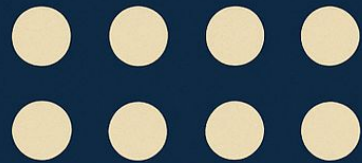
BOSCH



Ex. 3.1: Nav2



Ex. 3.1: Turtlebot

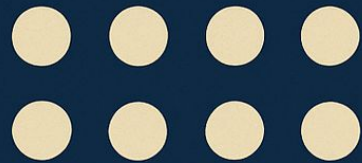


To install turtlebot:

```
sudo apt install ros-jazzy-turtlebot3*
```



Ex. 3.1: Turtlebot



We need to specify the type of robot we want to use:

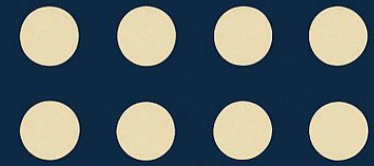
- `echo 'export TURTLEBOT3_MODEL=waffle' >> ~/.bashrc`
- `source ~/.bashrc`



Ex. 3.1



Ex. 3.1: Turtlebot



Write

A new environment variable

Where

- `echo 'export TURTLEBOT3_MODEL=waffle' >> ~/.bashrc`

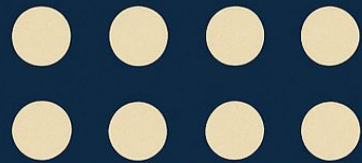
- `source ~/.bashrc`

System
environment
variable file

Execute



Ex. 3.1: Turtlebot

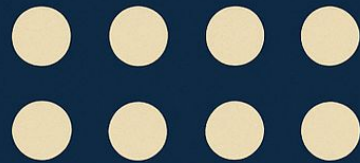


What we're doing exactly?

- We have a robot in an unknown environment
- Let's move around and create the map
- After this, we'll see how Nav2 allows to navigate autonomously

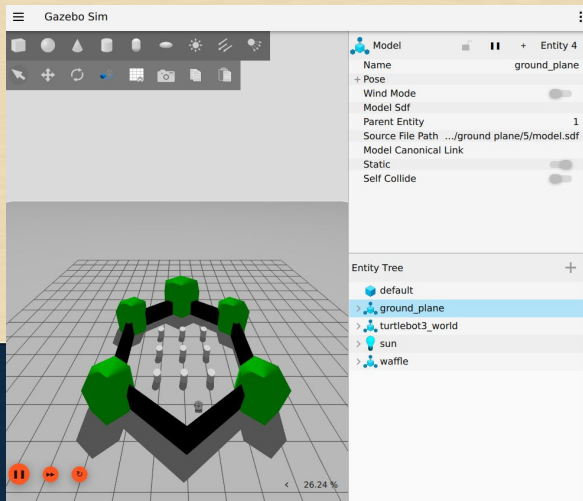


Ex. 3.1

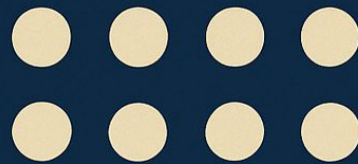


After entering **tmux** (**tmux**), first launch Gazebo:

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```



Ex. 3.1



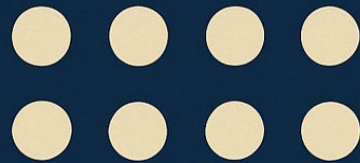
In a new terminal open RViz:

```
ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
```

We use the parameter `sim_time` because we are running a simulation, not a real robot



Ex. 3.1

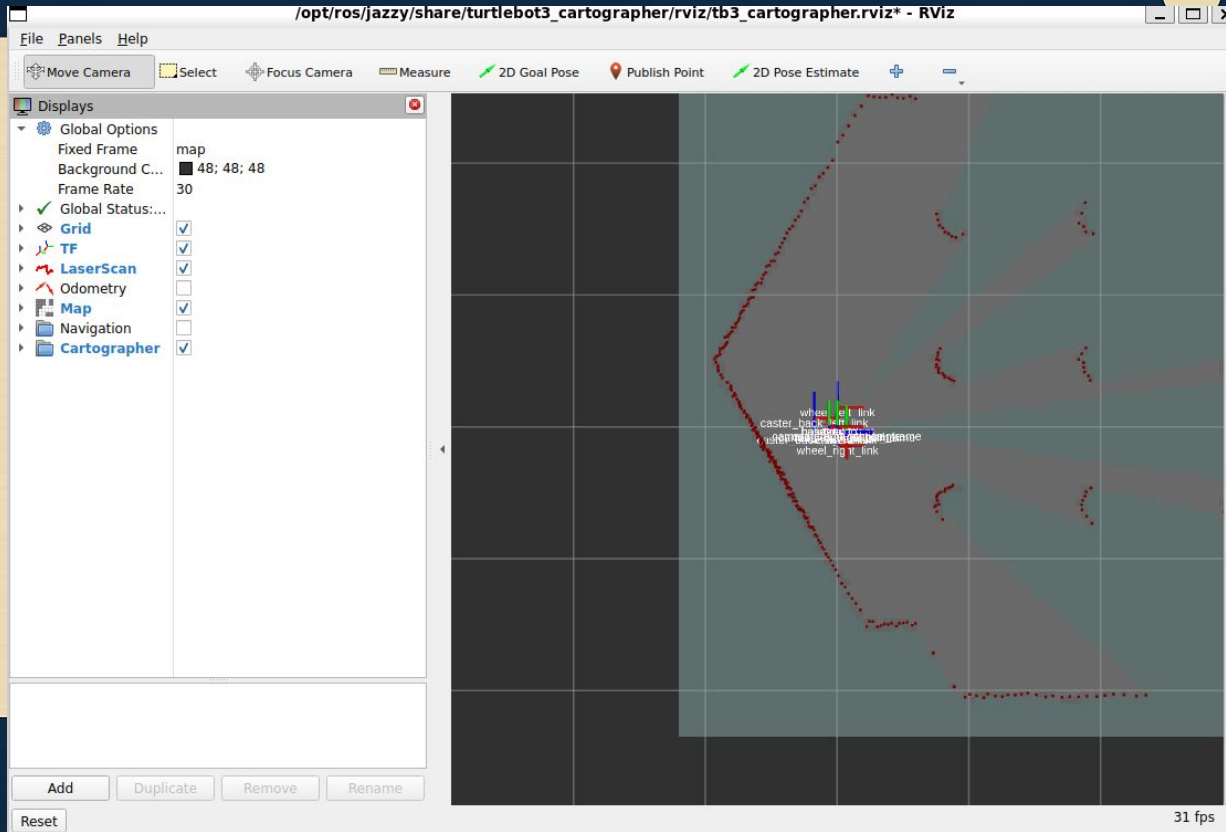


You will see the robot inside the environment.

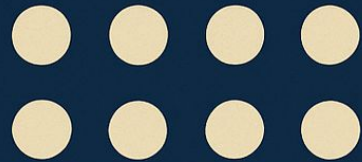
For better clarity we suggest to turn off the “Show names” parameter in TF.



Ex. 3.1



Ex. 3.1



To move the robot, in a new terminal:
ros2 run turtlebot3_teleop teleop_keyboard
The commands are
a bit more
complicated
than last time:

```
Control Your TurtleBot3!
```

```
-----  
Moving around:
```

```
      w  
a      s      d  
      x
```

```
w/x : increase/decrease linear velocity (Burger : ~ 0.22, Waffle and Waffle  
Pi : ~ 0.26)
```

```
a/d : increase/decrease angular velocity (Burger : ~ 2.84, Waffle and Waffle  
Pi : ~ 1.82)
```

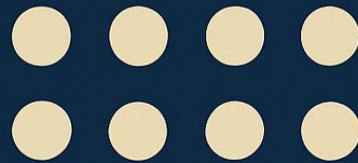
```
space key, s : force stop
```

```
CTRL-C to quit
```

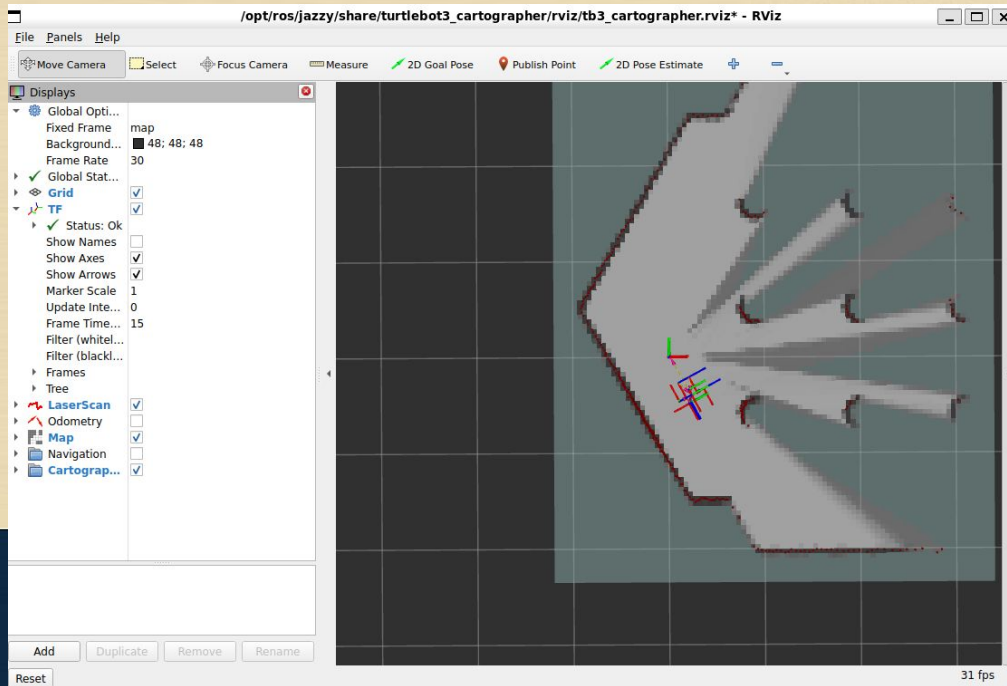
```
|  
[0] 0:python3*Z
```

```
"VictusLuca" 16:19 14-May-25
```

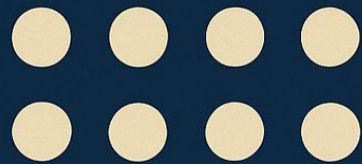
Ex. 3.1



After moving around, you'll see the map starts forming:



Ex. 3.1



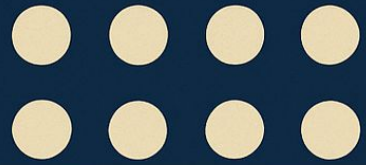
Once we have the full map, to save it open a new terminal **(without closing the running ones!)** and run:

```
ros2 run nav2_map_server map_saver_cli -f map_name
```

-f specifies the path where you want to save it



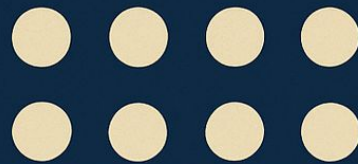
Ex. 3.2



Check that the map was saved correctly.
Now we can use the map as a “sandbox” to
navigate in. First stop everything
(ctrl+c), then let’s close everything
(to close a terminal, ctrl/b + x, then y)



Ex. 3.2

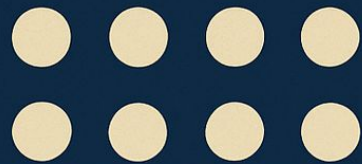


Let's start the robot again with the same command:

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```



Ex. 3.2



“Upload” the map into RViz, by starting Nav2 with the map as an argument. IN a new terminal:

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py \  
use_sim_time:=True map:=<path/to/map_name>.yaml
```

(NB: insert your map path here)



Ex. 3.2

The screenshot shows the RViz interface for a TurtleBot3 navigation setup. The main display area shows a top-down view of a navigation environment with a white polygonal area on a dark grid, containing several small robot icons. The interface includes various panels for displays, navigation, and docking.

Displays Panel:

- Global Options
 - Fixed Frame: map
 - Background Color: 48; 48; 48
 - Frame Rate: 30
- Global Status: Error
- Grid: ☒
- RobotModel: ☒
- TF: ☒
- LaserScan: ☒
- Bumper Hit: ☒
- Map: ☒
- Amcl Particle Swarm: ☒
- Global Planner: ☒
- Controller: ☒
- Realsense: ☒
- MarkerArray: ☒

Navigation 2 Panel:

- Navigation: unknown
- Localization: unknown
- Feedback: unknown
- Poses remaining: 0
- ETA: 0 s
- Distance remaining: 0.00 m
- Time taken: 0 s
- Recoveries: 0
- Buttons: Pause, Startup, Waypoint / Nav Through Poses Mode
- Tools for WP-Following: Save WPs, Load WPs, Pause WP
- Num of loops: 0, Store initial_pose: ☐
- Reset button
- Legend: Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click: Zoom. Shift: More options.

Views Panel:

- Type: TopDownOrtho (rviz_default_)
- Zero button
- Current View: TopDownOrtho (rviz_default_)
- Parameters: Near Clip: 0.01, Target F...: <Fixed Frame>, Scale: 95.1665, Angle: -1.5758, X: 1.32914, Y: 0.307783
- Buttons: Save, Remove, Rename

Docking Panel:

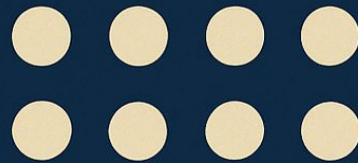
- Feedback: unknown
- State: none
- Time taken: 0 s
- Retries: 0
- Nav. to staging pose: ☒
- Dock id: ☒ []
- Dock type: Default
- Dock pose {X: [] Y: [] θ : []}
- Buttons: Dock robot, Undock robot

Selector Panel:

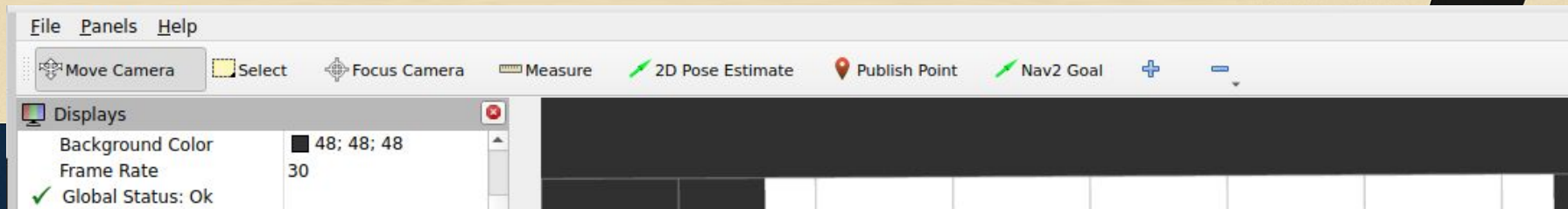
- Controller: Default
- Planner: Default
- Goal Checker: Smoother
- Progress Checker: Default

Status Bar: 31 fps

Ex. 3.2



The first thing to do is give our robot an estimate of its initial position. To do so, select “2d pose estimate” and try to click as close as possible to the real pose of the robot.



Ex. 3.2

RViz window showing a 2D navigation environment. The main display area shows a top-down view of a robot (blue square) navigating through a complex, multi-colored environment (purple, blue, green, yellow, red). The environment is bounded by a thick black line. The robot is currently positioned in the lower right quadrant, moving towards the center.

Displays Panel:

- Background Color: 48; 48; 48
- Frame Rate: 30
- Global Status: OK
- Fixed Frame: OK
- Grid: ☒
- RobotModel: ☐
- TF: ☒
- LaserScan: ☒
- Bumper Hit: ☒
- Map: ☒
- Amcl Particle Swarm: ☒
- Global Planner: ☒
- Controller: ☐
- Realsense: ☐
- MarkerArray: ☒

Navigation 2 Panel:

Navigation: active

Localization: active

Feedback: unknown

Poses remaining: 0

ETA: 0 s

Distance remaining: 0.00 m

Time taken: 0 s

Recoveries: 0

Buttons: Pause, Reset, Waypoint / Nav Through Poses Mode

Tools for WP-Following: Save WPs, Load WPs, Pause WP

Num of loops: 0 ☐ Store initial_pose

Reset

Views Panel:

Type: TopDownOrtho (rviz_default_)

Current View: TopDownOrtho (rviz_default_)

Near Clipping: 0.01

Target Fov: <Fixed Frame>

Scale: 95.1665

Angle: -1.5758

X: 1.32914

Y: 0.307783

Buttons: Save, Remove, Rename

Docking Panel:

Feedback: unknown

State: none

Time taken: 0 s

Retries: 0

Nav. to staging pose: ☒

Dock id:

Dock type: Default

Dock pose {X: Y: θ :

Buttons: Dock robot, Undock robot

Selector Panel:

Controller: Default

Planner: Default

Goal Checker: Default

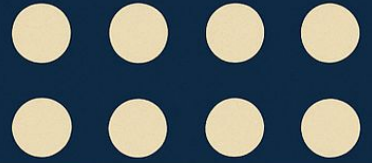
Smoother: Default

Progress Checker: Default

31 fps

Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click: Zoom. Shift: More options.

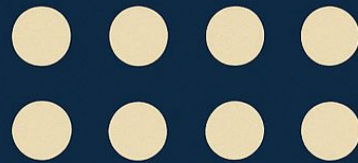
Ex. 3.2



You can clear up your view by unchecking the Global planner and the controller. Now the robot is ready to move! Try giving him a navigation goal with the command Nav2 goal



Q&A

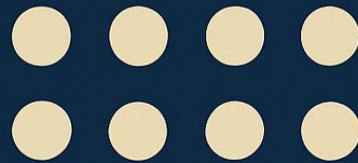


Why source the environment?

- So that your shell knows where to find ROS tools, libraries and packages
- It sets environment variables like `ROS_DISTRO` (tells your system which ROS version you're using)



Q&A



Can it be made automatic?

- Yes: add the source command `source /opt/ros/jazzy/setup.bash` to your `~/.bashrc` file. In this way, it gets sourced automatically every time you open a new terminal



Q&A

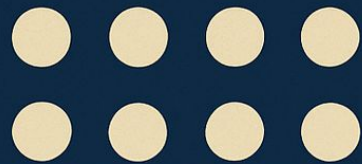


MY CHILD
WIL SOURCE
THE ENVIRONMENT
EVERY TIME!

MODIFY
~/.BASHRC



Q&A

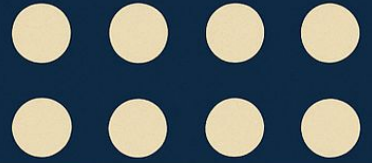


Every time I create a package, I have to add the dependencies manually?

- Not necessarily: you can add them when you create the package adding the parameter **--dependencies** to the `ros2 pkg create` command



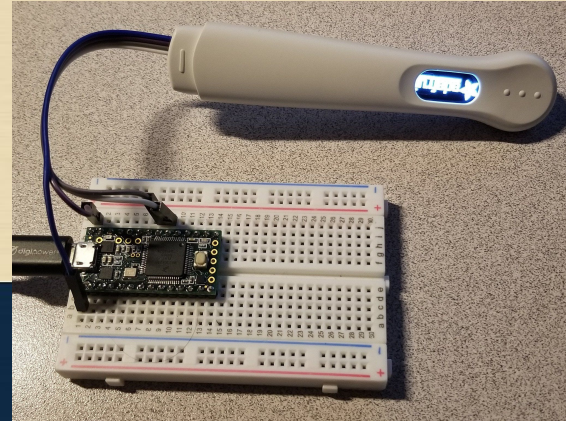
Q&A



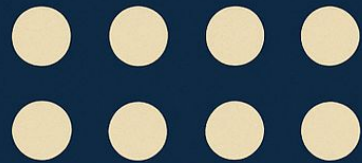
The most important question:
Can ROS run DOOM?



Can ROS run DOOM?



Can ROS run DOOM?

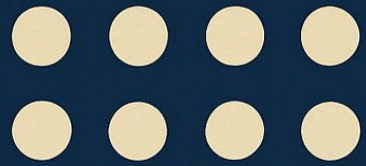


The answer is, of course, yes.
If you want to see for yourself:

https://github.com/gstavrinovs/doom_ros



Useful pckg to know/try



Moveit: if you want to play around with robotic arms

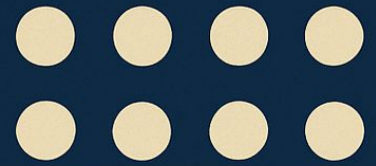
<https://moveit.picknik.ai/main/index.html>

microROS: if you're a microcontroller fan

<https://micro.ros.org/>



Useful pckg to know/try



You can find a package/repository for almost everything here:

https://index.ros.org/?search_packages=true

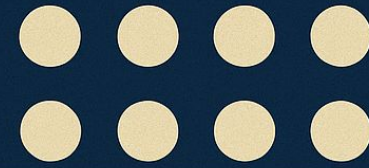


Future of Ros ?

New distribution just released (kinda)
Future support mainly for Humble, Jazzy and Kaiju



ROS WORKSHOP



Thank you!

